TeamUp — Walkthrough Video Script

Hack4Hack Crew Hiring Challenge 2026 · Problem Statement 1 · Target length: **4:00** (limit is 5:00 — this leaves a 1-minute buffer)

Before you hit record — a 5-minute setup checklist

- Open two browser windows/tabs, both logged in on the live app (`https://teamup-three-tau.vercel.app`) as two different demo accounts that are NOT currently in a team (e.g. demo2 and demo3 — ask Claude to check which ones are free right now).
- Open a terminal in the `server` folder, ready to run the race demo. Have this exact command typed but NOT yet run:
  ```
  TARGET=https://hack4hack-entry-production.up.railway.app MONGODB_URI="<Singapore connection string>" npm run race-demo
  ```
- Separately, have `docker compose up` already started in another terminal a minute or two before recording, so segment 7 just shows it already serving — don't make the viewer wait for a cold boot on camera.
- Have VS Code open to `server/src/services/teamService.js`, scrolled to the `addMemberInTxn` function, for segment 5.
- Have the GitHub Actions tab open (all green checks) for the last segment.
- Do one full dry run without recording first — it tells you if any line feels awkward out loud.

How to read this script

Each block below is one segment. **ON SCREEN** tells you exactly what to click to or show before you speak. **SAY** is a natural-sounding script — read it as a guide, not word-for-word; it's timed to fit the window given. Practice out loud twice before recording — it will feel faster once you know it.

SEGMENT 1 · 0:00–0:20 — 20 sec

Hook & the hard requirement

ON SCREEN

The live board, already logged in as a demo user, right after it loads.

SAY

Hi, I'm Pawan. This is TeamUp — a race-proof team-formation app built for the Hack4Hack Crew Hiring Challenge, Problem Statement 1. The hardest requirement: when fifty people click “join” on the very last open seat at the exact same instant, exactly one of them should win. I'm going to show you that's really true — live, on the deployed app, not a local demo.

SEGMENT 2 · 0:20–0:50 — 30 sec

Tour the live board

ON SCREEN

Scroll slowly down the board: the countdown, the “most wanted skills” bar chart, the stats strip, then the team/people cards.

SAY

This is the live board. It shows a countdown to the team-formation deadline, the skills teams want most right now across ten thousand seeded users and two thousand teams, and live counts of open seats and people looking. Every card here updates instantly over WebSockets — there is no refresh button anywhere in this app.

SEGMENT 3 · 0:50–1:30 — 40 sec

Create a team, invite, live join

ON SCREEN

Window 1: click “Create a team”, fill it in, create it, then “Generate invite code”. Switch to Window 2 (second demo user): paste the code, click Join. Switch back to Window 1.

SAY

Let's create a team and generate an invite code. Now, in a second browser as a different user, I'll join with that code. [switch windows, paste code, join] … And watch the first screen — no refresh — the member count just updated live, and the activity log shows exactly who joined and when.

SEGMENT 4 · 1:30–2:10 — 40 sec

THE race — the core requirement, live

ON SCREEN

Terminal: run the race-demo command you prepared. Let the output print fully on screen.

SAY

Now the real test. Fifty real users are about to fight for literally the last open seat of a live team — all fired at the exact same instant, over HTTP, against the deployed server you're looking at right now. [run the command, wait for output] … And there it is: forty-nine requests got “TEAM_FULL”, exactly one joined, and the integrity checker confirms the team's member count and leader are perfectly consistent. The database's own transaction decided the winner — no lock, no queue.

SEGMENT 5 · 2:10–2:40 — 30 sec

How it works, in one breath

ON SCREEN

VS Code: teamService.js, the conditional findOneAndUpdate line (openSeats: {$gt: 0}) already visible on screen.

SAY

Here's the trick: every join is one MongoDB transaction with a conditional update — take this seat only if open seats is greater than zero. Fifty requests hit the same document; the database commits exactly one and rejects the rest with a write conflict, and the losers retry with a jittered backoff. My first version used the driver's default retries, and one race like this needed over a thousand retries — tuning the backoff brought it down to forty-nine, one clean retry per loser.

SEGMENT 6 · 2:40–3:10 — 30 sec

Suggestions, audit log, accessibility

ON SCREEN

The suggestions panel with the scoring formula visible. Then quickly resize to phone width or tab through with the keyboard only.

SAY

Teammate suggestions are scored and explained — seventy percent on skills your team is missing, thirty percent on timezone overlap — nothing's a black box. The whole app also works on a phone, and entirely with just a keyboard, and passes an automated accessibility scan with zero serious violations.

SEGMENT 7 · 3:10–3:40 — 30 sec

Fully offline, one command

ON SCREEN

Terminal with docker compose up already running (started earlier) — cut straight to http://localhost:8080 loading and logging in.

SAY

And if a judge wants to run this with absolutely no cloud account: one command, docker compose up, boots a MongoDB replica set, seeds ten thousand users, and serves the whole app fully offline. I built this so nobody needs to create a single account to try it.

SEGMENT 8 · 3:40–4:00 — 20 sec

Tests, CI, and close

ON SCREEN

GitHub Actions page, all checks green.

SAY

Forty-five API tests, five browser tests, and a chaos test that kills the server mid-join and finds zero data corruption — all running in CI on every push. Thanks for watching — this is TeamUp.

Total: 8 segments, 4:00 exactly. If you run long anywhere, segment 6 (accessibility) or the phone-width cut in segment 2 are the safest to trim — everything else carries a required-feature proof.